

Migration Guide To Deprecate Nginx

Migration Guide: Deprecating Nginx (4.9.a)

In 4.9.a, the nginx ingress controller is removed. Traefik (introduced alongside nginx in 4.8.c) becomes the sole ingress controller. Most of the migration is automatic; there are three customer-visible touchpoints — one mandatory (DNS), two recommended hygiene (`values.yaml`).

What's automatic — no customer action

- **Cleanup of nginx Kubernetes resources.** Hubble configserver runs a one-shot script at 4.9.a startup that removes orphaned nginx Deployments, Services, RBAC, IngressClass, the `ingress-nginx` namespace (for non-Enterprise installs), and the `nginx-tls` Secret in application namespaces. No manual cleanup needed.
- **Enterprise TLS cert continuity.** For Enterprise installs, the cert-bridge introduced in 4.8.c is retained — it copies the uploaded TLS cert from the preserved nginx `default-ssl-certificate` Secret into the Traefik Secret on startup. Enterprise customers do not lose their uploaded cert.

What requires customer action

#	Change	Applies to	When	Action
1	Management-plane DNS record must point at the Traefik LoadBalancer	All install types, only when upgrading from pre-4.8.c	Post-upgrade	Update A / CNAME record — see DNS Migration below
2	<code>ingress.type</code> field removed from <code>values.yaml</code>	Helm-based installs (self-hosted, appliance)	Optional hygiene — silently ignored if left in	Remove from overrides if present

3	<code>ingress.ingress.internal</code> field removed from <code>values.yaml</code>	Helm-based installs (self-hosted, appliance)	Optional hygiene — silently ignored if left in	Remove from overrides if present
4	GKE Cloud IAM permissions for RBAC delete	Self-hosted GKE installs (helm-based)	Pre-upgrade	Grant the listed IAM permissions to the principal running <code>helm upgrade</code> . See \$4

1. DNS migration

Scenarios

Upgrading from	Current DNS state	Action
Pre-4.8.c	A record → nginx LoadBalancer	Update the A record to point at the Traefik LoadBalancer. Follow the DNS Cutover Procedure below.
4.8.c / 4.9.0	A record → Traefik LoadBalancer (already cut over)	No DNS action needed.

DNS Cutover Procedure

Step 1 — Capture the Traefik LoadBalancer address

```

1 kubectl get svc -n ingress-traefik traefik-ingress-controller
2
3 TRAEFIK_LB=$(kubectl get svc -n ingress-traefik traefik-ingress-controller \
4   -o jsonpath='{.status.loadBalancer.ingress[0].ip}{.status.loadBalancer.ingress[0].hostname}')
5 echo "Traefik LB: $TRAEFIK_LB"
6

```

Step 2 — Update the A record (or CNAME)

Choose your DNS provider.

AWS Route53 — Console

1. Navigate to **Route53** → **Hosted zones** → select your hosted zone.
2. Find the A record for the management-plane domain (e.g., `console.spectrocloud.com`).
3. Click **Edit record** and replace the **Value** with the Traefik LB IP/hostname from Step 1.
4. Save the record.

AWS Route53 — CLI

```
1 HOSTED_ZONE_ID="Z1234567890ABC"
2 DOMAIN_NAME="console.spectrocloud.com"
3 TRAEFIK_LB_IP="203.0.113.50"
4
5
6
7
8 aws route53 change-resource-record-sets \
9   --hosted-zone-id $HOSTED_ZONE_ID \
10  --change-batch '{
11    "Changes": [{
12      "Action": "UPSERT",
13      "ResourceRecordSet": {
14        "Name": "'$DOMAIN_NAME'",
15        "Type": "A",
16        "TTL": 60,
17        "ResourceRecords": [{"Value": "'$TRAEFIK_LB_IP'"}]
18      }
19    }]
20  }'
```

GCP Cloud DNS — Cloud Console → Network Services → Cloud DNS → select zone → edit the A record. CLI: `gcloud dns record-sets transaction start/add/execute`.

Azure DNS — Azure Portal → DNS Zones → select zone → edit the A record. CLI: `az network dns record-set a update`.

Cloudflare / NS1 / other managed DNS — provider's UI or API. Same A-record update.

On-prem / corporate DNS — update the record in the zone file on the authoritative server (BIND, Windows DNS, etc.) and reload. Coordinate with your DNS administrator.

If the domain uses a CNAME pointing at an LB hostname (common for AWS ELB), update the CNAME value instead.

Step 3 — Verify

```
1 # Should resolve to the Traefik LB
2 nslookup console.spectrocloud.com
3
4 # HTTPS reachability through the Traefik path
5 curl -k -I https://console.spectrocloud.com/
6
```

Expected: `nslookup` returns the Traefik LB IP/hostname; `curl` returns an HTTP response (200 / 301 / 302).

2. Helm `values.yaml` field changes

Applies to **helm-based installs** (self-hosted and appliance). SaaS and EC manifests-based installs are unaffected — they don't use `values.yaml`.

Two ingress-related fields are removed in 4.9.a. Both are silently ignored if present in your overrides — `helm upgrade` succeeds without warning. Removing them is recommended hygiene only; it has no functional effect.

`ingress.type`

Selected which ingress controller to install — `"nginx"` or `"traefik"`. Defaulted to `"traefik"` since 4.8.c. In 4.9.a, Traefik is the sole ingress controller, so the switch no longer has any meaning. Remove from your overrides if present.

`ingress.ingress.internal`

An nginx-specific toggle that put the nginx ingress controller into hostNetwork mode (instead of fronting it with a cloud LoadBalancer). Since nginx is no longer shipped in 4.9.a, the field no longer applies. Remove from your overrides if present.

The equivalent capability for Traefik — running on host ports instead of behind a LoadBalancer — is controlled by `ingress.traefik.hostPort`. This field is managed by the appliance build process and is not intended for customer

configuration; you should not need to set it manually.

4. GKE — Cloud IAM permissions

Applies to **self-hosted GKE installs only**. During `helm upgrade`, the principal running the upgrade must hold the following GKE Cloud IAM permissions so helm can prune nginx RBAC resources:

- `container.roles.delete`
- `container.roleBindings.delete`
- `container.clusterRoles.delete`
- `container.clusterRoleBindings.delete`

Reference: [PEM-10809](#).

5. Troubleshooting — configserver stuck on `init-rootdomain-traefik`

Applies to **Enterprise installs only**. This is a recovery procedure, not a required upgrade step. Run it only if you encounter the symptom below.

Symptom

After upgrading to 4.9.a, the `configserver` pod stays in `Init:CrashLoopBackOff` for the `init-rootdomain-traefik` init container, and its logs contain:

```
1 | level=fatal msg="not able to get the host ip. got NotFound: Resource 'websecure' not found of type 'pod'"
2 |
```

Verify with:

```
1 | kubectl -n hubble-system get pod -l component=configserver
2 | kubectl -n hubble-system logs <configserver-pod> -c init-rootdomain-traefik --previous | tail
3 | kubectl -n ingress-traefik get pods
4 |
```

If the traefik DaemonSet pods are stuck `Pending` with scheduler message `didn't have free ports for the requested pod ports`, leftover nginx-controller pods on the same nodes are holding hostPorts 80/443/9090. The automatic cleanup script

could not run because configserver itself never reached Ready. Clean up nginx manually using the steps below — once nginx is gone, traefik will schedule, the configserver init will succeed, and the rest of the cleanup will complete on its own on the next configserver start.

⚠ Do NOT delete Secret/default-ssl-certificate in the ingress-nginx namespace and do NOT delete the ingress-nginx namespace itself. Both are required by the Enterprise cert-bridge that continues to copy your uploaded TLS cert into Traefik on every configserver restart. Deleting them will break ingress TLS.

Step 1 — Delete cluster-scoped nginx resources

```
1 kubectl delete validatingwebhookconfiguration ingress-nginx-admission --ignore-not-found
2 kubectl delete clusterrole ingress-nginx ingress-nginx-admission --ignore-not-found
3 kubectl delete clusterrolebinding ingress-nginx ingress-nginx-admission --ignore-not-found
4 kubectl delete ingressclass nginx --ignore-not-found
5
```

Step 2 — Delete nginx resources inside the ingress-nginx namespace (keep the namespace and default-ssl-certificate)

```
1 kubectl delete daemonset ingress-nginx-controller -n ingress-nginx --ignore-not-found
2 kubectl delete service ingress-nginx-controller -n ingress-nginx --ignore-not-found
3 kubectl delete service ingress-nginx-controller-admission -n ingress-nginx --ignore-not-found
4 kubectl delete configmap ingress-nginx-controller -n ingress-nginx --ignore-not-found
5 kubectl delete serviceaccount ingress-nginx ingress-nginx-admission -n ingress-nginx --ignore-not-found
6 kubectl delete role ingress-nginx ingress-nginx-admission -n ingress-nginx --ignore-not-found
7 kubectl delete rolebinding ingress-nginx ingress-nginx-admission -n ingress-nginx --ignore-not-found
8 kubectl delete secret ingress-nginx-admission -n ingress-nginx --ignore-not-found
9 kubectl delete lease ingress-nginx-leader -n ingress-nginx --ignore-not-found
10 kubectl delete certificate ingress-nginx-admission -n ingress-nginx --ignore-not-found
11
```

Step 3 — Delete legacy nginx-tls Secret and nginx Certificate in application namespaces

```
1 for ns in hubble-system ui-system cp-system; do
2   kubectl delete certificate nginx -n $ns --ignore-not-found
3   kubectl delete secret nginx-tls -n $ns --ignore-not-found
4 done
5
```

Step 4 — Verify cleanup

```
1 # Cluster-scoped — all should return NotFound
```

```
2 kubectl get validatingwebhookconfiguration ingress-nginx-admission
3 kubectl get clusterrole ingress-nginx ingress-nginx-admission
4 kubectl get clusterrolebinding ingress-nginx ingress-nginx-admission
5 kubectl get ingressclass nginx
6
7 # Namespace must STILL exist
8 kubectl get namespace ingress-nginx
9
10 # Namespace contents - all should return NotFound
11 kubectl get daemonset ingress-nginx-controller -n ingress-nginx
12 kubectl get service ingress-nginx-controller -n ingress-nginx
13 kubectl get service ingress-nginx-controller-admission -n ingress-nginx
14 kubectl get configmap ingress-nginx-controller -n ingress-nginx
15 kubectl get serviceaccount ingress-nginx ingress-nginx-admission -n ingress-nginx
16 kubectl get role ingress-nginx ingress-nginx-admission -n ingress-nginx
17 kubectl get rolebinding ingress-nginx ingress-nginx-admission -n ingress-nginx
18 kubectl get secret ingress-nginx-admission -n ingress-nginx
19 kubectl get lease ingress-nginx-leader -n ingress-nginx
20 kubectl get certificate ingress-nginx-admission -n ingress-nginx
21
22 # default-ssl-certificate Secret must STILL exist
23 kubectl get secret default-ssl-certificate -n ingress-nginx
24
25 # nginx Certificate + nginx-tls Secret - all should return NotFound
26 for ns in hubble-system ui-system cp-system; do
27     kubectl get certificate nginx -n $ns
28     kubectl get secret nginx-tls -n $ns
29 done
30
```